

PY101: Week 2 - Decisions and Loops

Lesson 1: Control Flow Overview: Decisions vs Repetition

Control flow allows your program to make choices or perform repetitive actions[cite: 5]. Selection (if/else) chooses a specific path based on a condition, while Repetition (loops) repeats code blocks multiple times[cite: 5].

Selection vs. Repetition: A Quick Cheat Sheet

Think of **Selection** as an "If this, then that" branching path in your code. Think of **Repetition** (loops) as a "Do this until finished" conveyor belt. You use selection to route the program flow, and repetition to process items in bulk.

Lesson 2: If/else: Syntax, Indentation, Mental Model

Python uses indentation to group code blocks under conditions. Every `if` and `else` statement must end with a colon (:), and the code following them must be indented[cite: 5].

Why does Python force indentation?

In many other languages, blocks are defined by curly braces `{ }`. Python uses indentation instead because it makes the code look exactly like it behaves: the code inside an `if` block visually "belongs" to that condition because it is physically moved to the right.

Lesson 3: Logical Operators: and/or/not and Short-Circuiting

Logical operators allow for complex conditional checks. `and` requires all conditions to be true, `or` requires only one to be true, and `not` flips a boolean result[cite: 5]. Python uses **short-circuiting** to skip unnecessary checks if the final result is already determined[cite: 5].

What exactly is 'short-circuiting'?

Short-circuiting is an optimization. For example, in an `A and B` check, if `A` is already `False`, Python knows the whole statement must be `False`, so it doesn't even bother looking at `B`. This saves processing time and can prevent errors if `B` would have caused a crash.

Lesson 4: Booleans and Comparisons: Equality vs Identity

Booleans are values that are either `True` or `False`. When comparing, always use `==` for checking equality and `=` for variable assignment. While `is` checks if two objects are the same in memory, `==` is the preferred operator for comparing values[cite: 5].

Lesson 5: String Membership with 'in' in Conditions

The `in` operator checks if a substring exists within a larger string, returning a boolean value. It is highly useful for checking user input or filtering text patterns[cite: 5].

Wait, is 'in' case-sensitive?

Yes, `in` is case-sensitive! `"P" in "python"` would be `False` because the lowercase `"p"` does not match the uppercase `"P"`. If you need to check regardless of case, convert both strings to lowercase first using `.lower()`.

Lesson 6: Elif and Refactoring Nested Decisions

Use `if/elif/else` chains to flatten nested decision logic. Python processes these branches from top to bottom and runs only the *first* block that evaluates to `True`[cite: 5].

Why order matters in 'if/elif'

Python evaluates `if/elif` chains from top to bottom. It stops and runs the very first branch that evaluates to `True`. If you put a broad condition (like `score >= 60`) before a specific one (like `score >= 90`), the specific one will *never* run because the broad condition "catches" it first.

Lesson 7: Decision Tables to Branching Logic

Decision tables are a structured way to lay out conditions, ensuring logic is both **complete** (all cases covered) and **mutually exclusive** (no overlap between conditions)[cite: 5].

Lesson 8: For Loops and `range()`: Counting Iterations

The `range()` function generates a sequence of numbers. `range(start, stop, step)` is the standard pattern, where the `stop` value is exclusive[cite: 5].

The 'Stop' value rule

The `stop` value in `range()` is always exclusive, meaning the loop stops **before** it reaches that number. This is a common pattern in computer science because it makes calculating the total number of iterations easy: `range(start, stop)` will always run `stop - start` times.

Lesson 9: Meet the While Loop

Use a `for` loop when you know the number of iterations; use a `while` loop when you want to repeat until a specific condition changes[cite: 5]. Always ensure the loop variable changes inside the block to avoid an infinite loop[cite: 5].

How to kill an infinite loop

If you accidentally write a `while` loop that never ends (an infinite loop), your console will hang or print endlessly. In most code editors, you can press **Ctrl + C** to force the program to stop and kill the process.

Lesson 10: Loop Control: Break and Continue

`break` exits a loop entirely, while `continue` skips the remaining code in the current iteration and jumps to the next cycle[cite: 5].

Lesson 11: Counters and Totals

A counter tracks "how many" times an action occurs; a total accumulates values. Always initialize these variables (e.g., `total = 0`) *before* the loop starts to ensure they maintain their value across iterations[cite: 5].

Why initialize variables outside the loop?

If you put `total = 0` *inside* the loop, it resets to zero every single time the loop repeats. You would lose all your previous work! Initializing outside ensures the variable acts as a persistent container that survives across every iteration of the loop.